agency.py

```
import os
import json
from pathlib import Path
from dotenv import load_dotenv
from groq import Groq

load_dotenv(Path(__file__).resolve().parent / ".env")

client = Groq(api_key=os.getenv("GROQ_API_KEY"))
MODEL = "openai/gpt-oss-120b"



PRICES = {
    "shoes": 799,
    "hat": 399,
    "bag": 1420,
    "shorts": 1299,
    "pants": 1699,
}

def get_price(item):
    print(f"🔧 Tool called: get_price({item})")
    price = PRICES.get(item.lower())
    return f"₹{price}" if price is not None else f"Price not found for {item}"

tools = [{
    "type": "function",
    "function": {
        "name": "get_price",
        "description": "Get the price of a shop item.",
        "parameters": {
            "type": "object",
            "properties": {
                "item": {
                    "type": "string",
                    "description": "The item name"
                }
            },
            "required": ["item"],
```

```
            "additionalProperties": False
        }
    }
}]

def agent(user_message):
    messages = [{"role": "user", "content": user_message}]

    for _ in range(5):
        response = client.chat.completions.create(
            model=MODEL,
            messages=messages,
            tools=tools,
            tool_choice="auto"
        )

        msg = response.choices[0].message
        messages.append(msg)

        if not msg.tool_calls:
            return msg.content or ""

        for call in msg.tool_calls:
            if call.function.name != "get_price":
                result = "Unknown tool"
            else:
                args = json.loads(call.function.arguments)
                result = get_price(args["item"])

            messages.append({
                "role": "tool",
                "tool_call_id": call.id,
                "content": result
            })

    return "I couldn't complete the request after several tool calls."

if __name__ == "__main__":
    print(agent("How much are the shoes?"))
    print(agent("Hi! What can you help with?"))
```

explain the code of pythone line by line

Line-by-line explanation of `memory_demo.py`

First, one important clarification: your current code is not using LangChain or Grok. It is using the Groq Python SDK directly.

You are building a simple AI Agent with Tool Calling.

The agent can:

1. Receive a question from a user.
2. Understand whether it needs a tool.
3. Call the Python function `get_price()`.
4. Receive the tool's result.
5. Send that result back to the LLM.
6. Generate a natural-language response.

For example:

User: How much are the shoes?

AI Agent: Calls `get_price("shoes")`.

Python Tool: Returns `₹799`.

AI Agent: The price of shoes is ₹799.

1. Overall architecture

User

How much are the shoes?

Python Agent — agent()

Sends messages and tool definitions to Groq

Groq LLM

Decides whether to answer or call a tool

Python Tool: get_price(item)

Looks up the price in PRICES dictionary

Tool result: ₹799

Sent back to the Groq LLM

Final response to user

The shoes cost ₹799.

2. Import statements

Your code:

```
None
import osimport jsonfrom pathlib import Pathfrom dotenv import load_dotenvfrom groq import Groq
```

Let's understand each import.

Line 1:

```
None
import os
```

- Imports Python's built-in `os` module.
- Used to access environment variables.
- In your project, it retrieves the Groq API key.

Example:

```
None
os.getenv("GROQ_API_KEY")
```

Line 2:

```
None
import json
```

- Imports Python's JSON module.
- JSON stands for JavaScript Object Notation.
- Used to convert JSON strings into Python dictionaries.

Example:

```
None
args = json.loads(call.function.arguments)
```

Suppose the LLM returns:

```
None
{"item": "shoes"}
```

Python converts it into:

```
None
{"item": "shoes"}
```

Now you can access:

```
None
args["item"]
```

Result:

```
None
shoes
```

Line 3:

```
None
from pathlib import Path
```

- Imports the `Path` class.
- Helps work with file and folder paths.
- Provides a convenient, cross-platform way to locate your `.env` file.

Line 4:

```
None
from dotenv import load_dotenv
```

- Imports the `load_dotenv()` function.
- Reads variables from your `.env` file.
- Makes them available as environment variables.

Line 5:

```
None
from groq import Groq
```

- Imports the Groq client.
- Allows Python to communicate with Groq's hosted LLM API.

In simple words, this is how your Python application connects to the AI model.

3. Load environment variables and initialize Groq

Your code:

```
None
load_dotenv(Path(__file__).resolve().parent / ".env")client =
Groq(api_key=os.getenv("GROQ_API_KEY"))MODEL =
"openai/gpt-oss-120b"
```

Line 1:

```
None
load_dotenv(Path(__file__).resolve().parent / ".env")
```

Let's break this into smaller parts:

Expression	Meaning
`__file__`	Current Python file location
`.resolve()`	Gets its absolute path
`.parent`	Gets the containing folder
`/ ".env"`	Locates the `.env` file
`load_dotenv()`	Loads variables from that file

For example, your project:

```
None
AILearning2_grok/
    memory_demo.py
    .env
```

The program loads the `.env` file from the same folder as `memory_demo.py`.

Line 2:

```
None
client = Groq(api_key=os.getenv("GROQ_API_KEY"))
```

This creates a Groq API client.

- `os.getenv()` reads the API key.
- `Groq()` initializes the client.
- `client` is the object you use to make API requests.

Your `.env` file should contain:

```
None
GROQ_API_KEY=your_actual_groq_api_key
```

Line 3:

```
None
MODEL = "openai/gpt-oss-120b"
```

Specifies which model your application will request from Groq.

Note: Although the model name contains `openai`, you're accessing it through Groq's API, not OpenAI's API.

4. Product price dictionary

Your code:

```
None
PRICES = {    "shoes": 799,    "hat": 399,    "bag": 1420,
"shorts": 1299,    "pants": 1699,}
```

This is a Python dictionary.

A dictionary stores data in key-value pairs.

Key (item)	Value (price)
shoes	₹799
hat	₹399
bag	₹1,420
shorts	₹1,299
pants	₹1,699

For example:

```
None
PRICES["shoes"]
```

Output:

```
None
799
```

Why do we use a dictionary?

Because we want to retrieve the price of an item quickly using its name.

This is your application's local product database. It is not querying an external shopping website or database.

5. Define the Python tool: `get_price()`

Your code:

```
None
def get_price(item):    print(f"🔧 Tool called: get_price({item})")
price = PRICES.get(item.lower())    return f"₹{price}" if price is
not None else f"Price not found for {item}"
```

This is one of the most important parts of your AI agent.

Let's understand it line by line.

Line 1:

```
None
def get_price(item):
```

- `def` defines a Python function.
- `get_price` is the function name.
- `item` is the input parameter.

Example:

```
None
get_price("shoes")
```

Here, `item = "shoes"`.

Line 2:

```
None
print(f"🔧 Tool called: get_price({item})")
```

This prints a debugging message.

Output:

```
None
🔧 Tool called: get_price(shoes)
```

It helps you understand when the AI agent invokes your Python tool.

Line 3:

```
None
price = PRICES.get(item.lower())
```

This line performs the actual lookup.

Let's break it down:

- `item.lower()` converts the item name into lowercase.
- `PRICES.get()` searches for that key in the dictionary.
- The result is assigned to `price`.

Example:

```
None
item = "SHOES"price = PRICES.get(item.lower())
```

Output:

```
None
799
```

Why use `.lower()`?

So that `SHOES`, `Shoes`, and `shoes` all match the same dictionary key.

Line 4:

```
None
return f"₹{price}" if price is not None else f"Price not found for
{item}"
```

This is called a Python conditional expression (ternary operator).

It means:

- If the price exists, return the price with the rupee symbol.

- Otherwise, return a message saying the price was not found.

Equivalent longer version:

```
None
if price is not None:    return f"₹{price}"else:    return f"Price not found for {item}"
```

Example:

```
None
get_price("shoes")
```

Output:

```
None
₹799
```

Example:

```
None
get_price("watch")
```

Output:

```
None
Price not found for watch
```

6. Define the tool schema

Your code:

```
None
tools = [{    "type": "function",    "function": {        "name": "get_price",        "description": "Get the price of a shop item.",        "parameters": {            "type": "object",
```

```
"properties": {                "item": {                    "type":
"string",                   "description": "The item name"
}            },            "required": ["item"],
"additionalProperties": False        }    }}]
```

This is another very important concept: Function Calling / Tool Calling.

You have already created a Python function called `get_price()`.

But how does the LLM know that this function exists?

You provide the tool schema to the LLM.

Let's understand each part.

A. Tool type

```
None
"type": "function"
```

Tells the API that this tool is a function.

B. Function name

```
None
"name": "get_price"
```

Provides the function name to the LLM.

The LLM can request this function when the user asks about product prices.

C. Description

```
None
"description": "Get the price of a shop item."
```

Explains what the function does.

This description helps the LLM decide when to use the tool.

D. Parameters

```
None
"parameters": {    "type": "object",
```

The function accepts arguments represented as a JSON object.

E. Properties

```
None
"properties": {    "item": {        "type": "string",
"description": "The item name"    }}
```

This tells the LLM that the function expects one parameter:

- Name: `item`
- Data type: `string`
- Description: product name.

For example, the LLM may generate:

```
None
{"item": "shoes"}
```

F. Required

```
None
"required": ["item"]
```

The `item` parameter must be supplied.

G. Additional properties

```
None
"additionalProperties": False
```

Prevents extra, undeclared parameters from being included in the function arguments.

For example, the following is valid:

```
None
{"item": "shoes"}
```

But this is invalid under your schema:

```
None
{"item": "shoes", "quantity": 2}
```

Important distinction:

The tool schema does not execute your Python function. It only describes the function to the LLM.

Your Python code is responsible for actually executing it.

7. Main AI Agent function

Your code:

```
None
def agent(user_message):    messages = [{"role": "user", "content": user_message}]
```

This function manages the entire conversation with the LLM and your Python tool.

Line 1:

```
None
def agent(user_message):
```

Defines an agent function that accepts a user's question.

Example:

```
None
agent("How much are the shoes?")
```

Here:

```
None
user_message = "How much are the shoes?"
```

Line 2:

```
None
messages = [{"role": "user", "content": user_message}]
```

Creates the conversation history.

The `messages` variable is a Python list containing dictionaries.

Initially:

```
None
[    {        "role": "user",        "content": "How much are the shoes?"    }]
```

The `role` identifies who sent the message.

Role	Meaning
`user`	User's message
`assistant`	LLM's response
`tool`	Result returned by your Python function

This list will grow as the agent communicates with the LLM and executes tools.

8. Tool-calling loop

Your code:

```
None
for _ in range(5):
```

This creates a loop that allows up to five LLM interaction cycles.

Why?

Sometimes an LLM may request multiple tools or need another interaction after receiving a tool result.

The underscore `_` means you don't need the loop counter.

For example:

```
None
range(5)
```

Produces five iterations.

Next:

```
None
response = client.chat.completions.create(    model=MODEL,
messages=messages,    tools=tools,    tool_choice="auto")
```

This is where your application sends the request to Groq.

Let's understand each argument:

Argument	Purpose
`model=MODEL`	Selects the LLM
`messages=messages`	Sends conversation history

`tools=tools`	Provides available tool definitions
`tool_choice="auto"`	Allows the LLM to decide whether to call a tool

The LLM can now decide:

- Answer directly without calling a tool.
- Request the `get_price` function.

For example:

User asks:

```
None
How much are the shoes?
```

The LLM may request:

```
None
{"item": "shoes"}
```

Next:

```
None
msg = response.choices[0].message
```

Let's break it down:

- `response`: complete API response from Groq.
- `choices`: available model response choices.
- `[0]`: selects the first choice.
- `.message`: retrieves the assistant's message.

Next:

```
None
messages.append(msg)
```

Adds the assistant's message to the conversation history.

This is important because the next API request needs the context of the previous interaction.

9. Check whether the LLM requested a tool

Your code:

```
None
if not msg.tool_calls:    return msg.content or ""
```

This checks whether the LLM requested any tools.

`msg.tool_calls` contains the requested function calls, if any.

There are two possibilities.

Scenario A: No tool call

For example:

User:

```
None
Hi! What can you help with?
```

The LLM can answer directly.

Then:

```
None
msg.tool_calls
```

is empty.

Therefore:

```
None
return msg.content or ""
```

Returns the LLM's text response.

Scenario B: Tool call requested

For example:

```
None
How much are the shoes?
```

The LLM requests `get_price`.

Then `msg.tool_calls` is not empty, and the program continues to execute the tool.

10. Execute the requested tool

Your code:

```
None
for call in msg.tool_calls:
```

This loops through all tool calls requested by the LLM.

The LLM may request one or multiple tools.

Next:

```
None
if call.function.name != "get_price":    result = "Unknown tool"
```

Checks whether the requested function name is `get_price`.

If the function name is different, it returns:

```
None
Unknown tool
```

Next:

```
None
else:    args = json.loads(call.function.arguments)    result =
get_price(args["item"])
```

This is where the actual Python function gets executed.

Let's understand the two statements.

First:

```
None
args = json.loads(call.function.arguments)
```

The LLM provides function arguments as a JSON string.

Example:

```
None
{"item": "shoes"}
```

`json.loads()` converts that JSON string into a Python dictionary.

Now:

```
None
args["item"]
```

Returns:

```
None
shoes
```

Second:

```
None
result = get_price(args["item"])
```

Calls your actual Python function.

Equivalent to:

```
None
result = get_price("shoes")
```

Output:

```
None
🔧 Tool called: get_price(shoes)
```

And the returned value is:

```
None
₹799
```

So now:

```
None
result = "₹799"
```

11. Send the tool result back to the LLM

Your code:

```
None
messages.append({    "role": "tool",    "tool_call_id": call.id,
"content": result})
```

This is a very important step.

The Python tool has completed its execution, but the LLM still needs to know the result.

You add the result to the conversation history.

Let's understand the fields.

Field	Meaning
`role`	Identifies this as a tool response
`tool_call_id`	Connects the response to the original tool request
`content`	Contains the actual tool output

Example:

```
None
{    "role": "tool",    "tool_call_id": "call_123",    "content": "₹799"}
```

Now the conversation history contains:

1. User's question.
2. Assistant's tool request.
3. Tool's result.

The loop runs again and sends this updated conversation to Groq.

The LLM can now formulate a natural-language answer.

For example:

```
None
The price of shoes is ₹799.
```

This is the key concept of tool calling:

LLM decides → Python executes → LLM responds.

12. Maximum tool-call limit

Your code:

```
None
return "I couldn't complete the request after several tool calls."
```

This statement executes if the loop finishes all five iterations without returning a normal response.

It prevents the agent from running indefinitely.

13. Main entry point

Your code:

```
None
if __name__ == "__main__":    print(agent("How much are the shoes?"))    print(agent("Hi! What can you help with?"))
```

This is a standard Python execution pattern.

Line 1:

```
None
if __name__ == "__main__":
```

It checks whether this Python file is being executed directly.

If you execute:

```
None
python memory_demo.py
```

The condition is true.

If another Python file imports `memory_demo.py`, the condition is false.

Line 2:

```
None
print(agent("How much are the shoes?"))
```

Calls your agent with a product-price question.

Expected response:

```
None
🔧 Tool called: get_price(shoes)
The shoes cost ₹799.
```

The exact natural-language response depends on the LLM.

Line 3:

```
None
print(agent("Hi! What can you help with?"))
```

This sends a general question.

The LLM should answer directly without using the `get_price` tool.

For example:

```
None
Hi! I can help you check the prices of shoes, hats, bags, shorts,
and pants.
```

14. Complete execution flow with an example

Let's understand what happens internally when you execute:

```
None
agent("How much are the shoes?")
```

1. User asks a question
   `How much are the shoes?`
2. Python initializes messages
   `role=user`, `content=How much are the shoes?`
3. Python sends request to Groq
   Sends model, messages, tools and automatic tool choice.
4. LLM selects get_price
   Requests `{"item": "shoes"}`.
5. Python executes get_price()
   Searches `PRICES` and finds `799`.
6. Python sends tool result to Groq
   `role=tool`, `content=₹799`.
7. LLM generates final answer
   `The shoes cost ₹799.`
8. Python prints the response
   User sees the final answer.

15. Important concepts for your AI Engineering interviews

Concept	Meaning in your project
LLM	GPT-OSS-120B accessed through Groq
Agent	`agent()`
Tool	`get_price()`
Tool schema	`tools` dictionary
Tool calling	LLM requests a Python function

Tool execution	Python executes `get_price()`
Conversation history	`messages` list
JSON parsing	`json.loads()`
API client	`Groq()`
Environment variables	`.env` and `load_dotenv()`

16. Is this LangChain or LangGraph?

No. Your current implementation uses the Groq SDK directly.

Your current implementation

Direct Groq SDK

- You manually define tools.
- You manually manage conversation messages.
- You manually execute tool calls.
- You manually implement the loop.

LangChain implementation

Framework-based

- Uses LangChain model integrations.
- Provides tool abstractions.
- Provides agent-building capabilities.
- Simplifies message and tool orchestration.

LangGraph implementation

Graph-based orchestration

- Uses nodes and edges.

- Supports conditional routing.
- Supports state management.
- Supports more complex agent workflows.

17. Three improvements I recommend

1. Add input validation: Handle missing or invalid item names safely.
2. Add error handling: Handle API failures, invalid JSON and rate limits.
3. Add conversation memory: Currently, each call to `agent()` starts with a new `messages` list. Your second question does not remember the first question.

One more observation: despite the filename `memory_demo.py`, your current code does not implement persistent memory. It implements a stateless tool-calling agent.

Practice questions

Try answering these three senior-level AI engineering questions:

1. Who actually executes the get_price() function?

Groq LLM
Python application
LangChain
JSON parser

Correct answer: Python application

The LLM only requests the function. Your Python application executes it.

2. Why do we append the tool result to messages?

To save it in a database
To train the LLM
To provide the result back to the LLM
To create a new API key

Correct

The LLM needs the tool result in the conversation to generate its final answer.

3. Does the current agent remember previous calls to agent()?

Yes, automatically
No, every call creates a new messages list
Yes, through Groq

Only when using JSON

Correct answer: No, every call creates a new messages list

Each call initializes a fresh messages list. Persistent or shared conversation memory is not implemented.

Check answers

Your score: 1/3

Next, I can help you convert this exact program into a LangChain tool-calling agent using your existing Groq API key, keeping the same `PRICES` dictionary and `get_price()` function.